Модели на разпредели на разпределените софтуерни архитектури

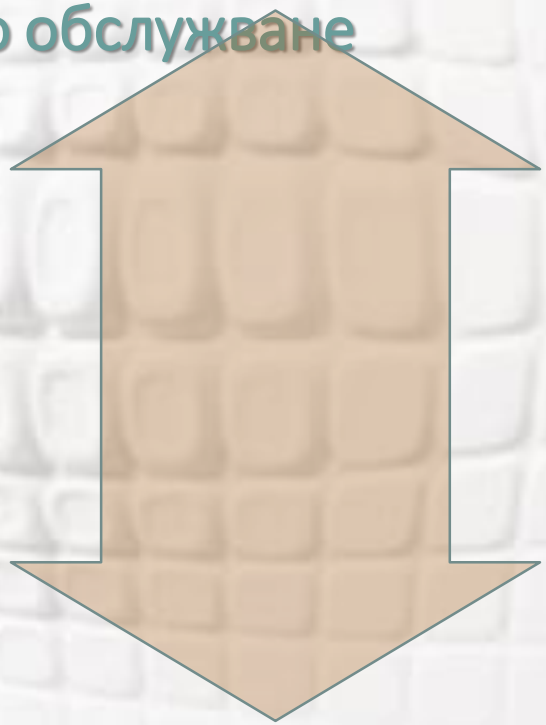
въпрос 15. /ДИ-СИ

ВАСИЛ ГЕОРГИЕВ

v.georgiev@fmi.uni-sofia.bg

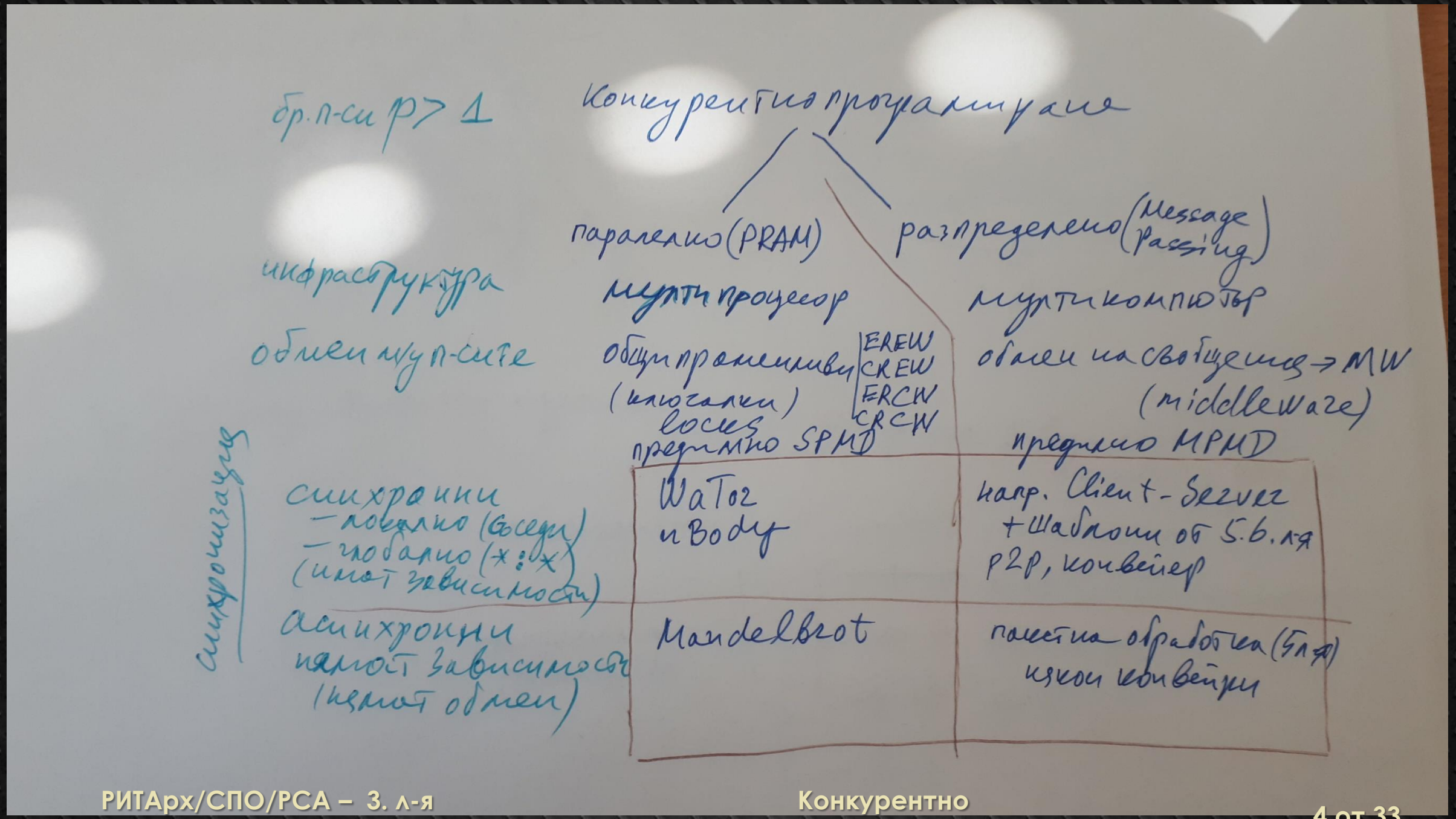
Модели на разпределените софтуерни/ИТ архитектури

- Параметри на конкурентната (разпределена или паралелна) обработка
- Модели PCA
  - неспециализирани – процедурни и обектни и
  - специализирани – потокови, контекстни, йерархични, асинхронни и интерактивни.
- Организация на разпределените приложения за масово обслужване
  - Сървени модели
    - Клиент-сървер
    - Трислоен модел
    - Брокерен модел
    - Сервизно-базиран модел
    - Компонентно-базиран моделБезсървени модели (p2p)
  - Безсърверно обслужване (p2p)
  - Синхронизация
    - протоколи за подреждане на съобщенията
    - разпределени транзакции



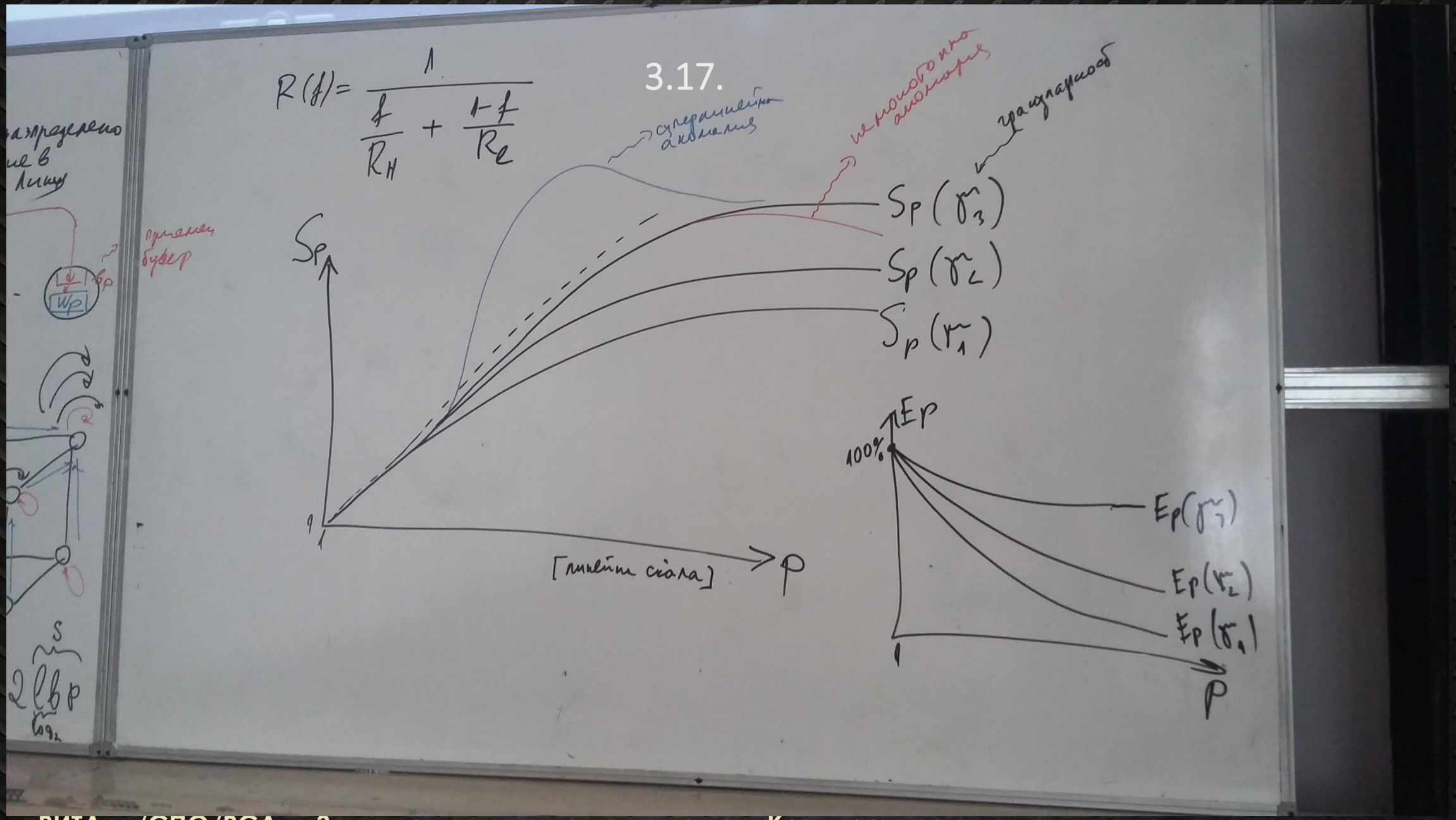
Параметри на конкурентната (разпределена или паралелна) обработка

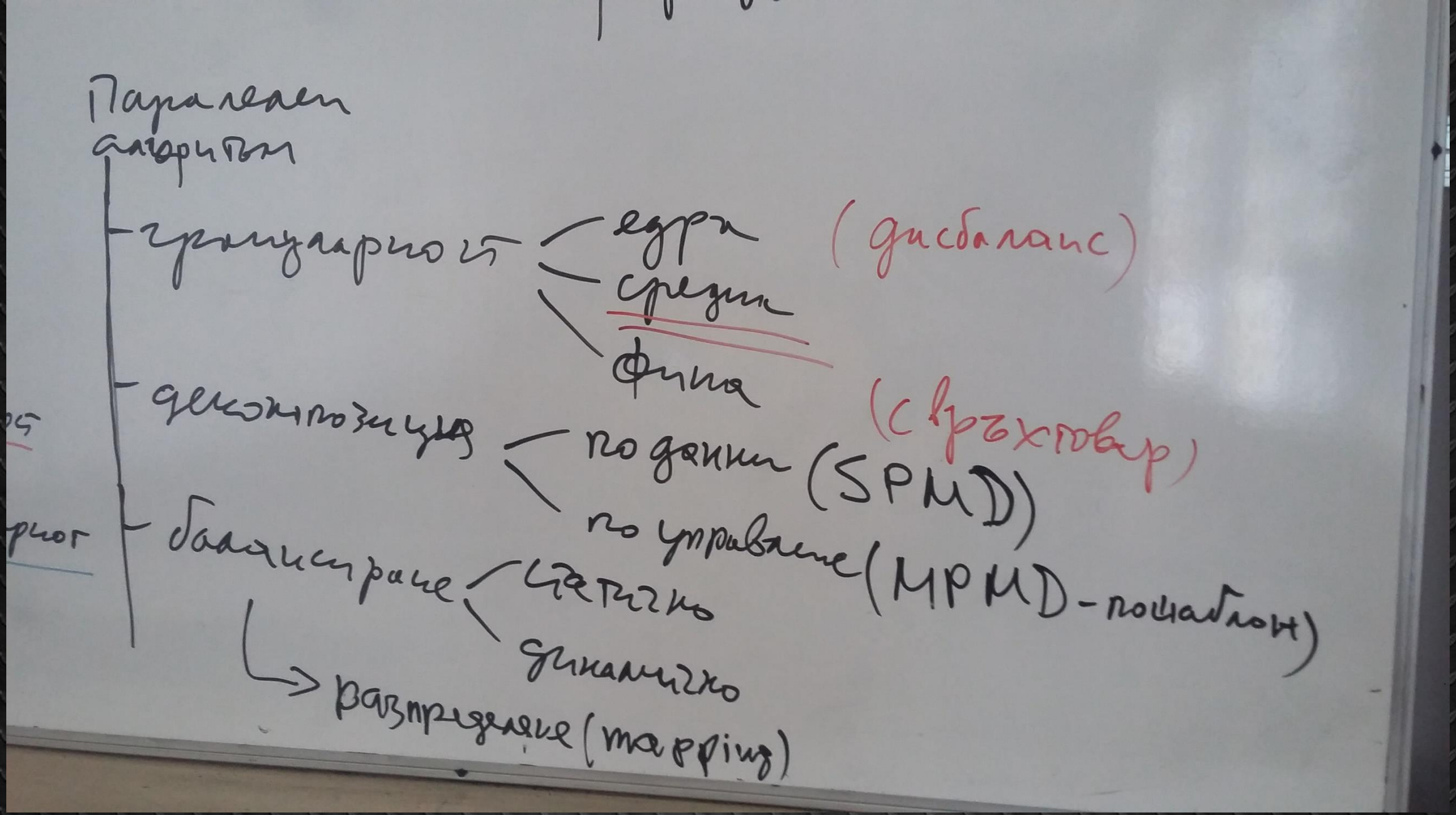
- софтуерен и хардуерен паралелизъм: *p* – брой процеси/нишки респ. брой ядра/възли (машини)
- тип обработка – паралелна или разпределена
- ускорение и ефективност S_p и E_p
- грануларност – фина, средна, едра
- синхронизация – асинхронни, локално синхронни, глобално синхронни
- балансиране между процесите – централизирано (топология звезда) или разпределено (напр. верига, решетка, хиперкуб), йерархично (клъстери – дърво)


бр. п-си P > 1
Конкурентно програмиране
инфраструктура
обмен м/у п-сите
паралелно (PRAM)
разпределено (Message Passing)
мултипроцесор
мултикомпютър
общи променливи (ключалки)
EREW
CREW
ERCW
CRCW
обмен на съобщения → MW (middleware)
basics
предимно SPMD
предимно MPMD
синхронизация
синхронни
– локално (съседи)
– глобално (*:*)
(имат зависимости)
WaTor и Body
напр. Client-Server + Шаблони об. 5.б.л-я P2P, конвейер
асинхронни
нямат зависимости
(нямат обмен)
Mandelbrot
пакетна обработка (бл-я) някои конвейери

УСКОРЕНИЕ И ЕФЕКТИВНОСТ

- ПРИ ОЦЕНКА ИЛИ ИЗМЕРВАНЕ НА УСКОРЕНИЕТО ($S_p = T_1/T_p$) СЕ ПРИЕМА, ЧЕ ВСИЧКИ ПРОЦЕСОРИ В ДВАТА СЛУЧАЯ СА С ИДЕНТИЧНА ПРОИЗВОДИТЕЛНОСТ; ПОРАДИ НАЛИЧИЕ НА КОМУНИКАЦИОННИ И СИНХРОНИЗАЦИОННИ ЗАКЪСНЕНИЯ $1 < S_p < p$
- СУПЕРЛИНЕЙНА АНОМАЛИЯ: $S_p > p$ МОЖЕ ДА СЕ НАБЛЮДАВА ПРИ НЕОПТИМАЛЕН ПОСЛЕДОВАТЕЛЕН АЛГОРИТЪМ ИЛИ ПРИ ОСОБЕНИ ХАРАКТЕРИСТИКИ НА ПРОБЛЕМА, КОИТО ИЗЯВЯВАТ НИСЪК КАПАЦИТЕТ НА ИЗПОЛЗВАНИЯ ХАРДУЕР: НАПР. ПРИ ГОЛЯМ РАЗМЕР НА ДАННИТЕ (НАДВИШАВАЩ КАПАЦИТЕТА НА ОП) Е ВЪЗМОЖНО ЗНАЧИТЕЛНО ЗАКЪСНЕНИЕ НА ПОСЛЕДОВАТЕЛНАТА ОБРАБОТКА НА ПРОБЛЕМА ПОРАДИ БАВНИ ОПЕРАЦИИ С ВЪНШНАТА ПАМЕТ, ДОКАТО ПРИ ПАРАЛЕЛНА ОБРАБОТКА С РАЗДЕЛЯНЕТО НА ДАННИТЕ МЕЖДУ ВЪЗЛИТЕ ТОЗИ ПРОБЛЕМ ОТПАДА (ОПТИМИЗИРАНЕ НА ПОСЛЕДОВАТЕЛНИЯ АЛГОРИТЪМ В ТОЗИ СЛУЧАЙ НЕ Е ВЪЗМОЖНО КОГАТО ЗА ДАННИТЕ НЕ ВАЖИ ПРИНЦИПА НА ЛОКАЛНОСТ – НАПР. ПРИ МНОГО ПРОБЛЕМИ ОТ AI)
- НЕМОНОТОННА АНОМАЛИЯ: $\exists p > p_{sat}: S_p > S_{p+1}$
- ЕФЕКТИВНОСТТА, КОЯТО Е НОРМАЛИЗИРАНО УСКОРЕНИЕ ($E_p = S_p/p = T_1/(pT_p) < 100\%$), ХАРАКТЕРИЗИРА ЧАСТТА ОТ ОБЩОТО ВРЕМЕ ЗА ПАРАЛЕЛНА ОБРАБОТКА, ПРЕЗ КОЯТО ПРОЦЕСОРНИТЕ ЕЛЕМЕНТИ СЕ ИЗПОЛЗВАТ ЕФЕКТИВНО


3.17.
$R(f) = \frac{1}{\frac{f}{R_H} + \frac{1-f}{R_L}}$
суперлинейна аномалия
незначителна аномалия
градиентна буфер
$S_P(\gamma_3)$
$S_P(\gamma_2)$
$S_P(\gamma_1)$
S_P
[линейна скала]
ρ
E_P
100%
$E_P(\gamma_3)$
$E_P(\gamma_2)$
$E_P(\gamma_1)$
ρ


Паралелен алгоритъм
гранулярност
едра
средна
фина
(дисбаланс)
декомпозиция
по данни
по управление
(SPMD)
(MPMD - по шаблон)
балансиране
статично
динамично
разпределяне (mapping)

Категории потокови архитектури

- топологията на пренос на данните между модулите се задава експлицитно с блок-диаграми
- модулите поддържат само интерфейс по данни, не и контролен интерфейс и не се адресират взаимно – адресацията е само чрез предаваните данни
- по механизма на свързване между модулите (т.е. на обмен) се разграничават
  - пакетна обработка (Batch Sequential) – отложена, цел оптимално използване на ресурси
  - филтрирани канали (Pipe & Filter) – конвейер, интерактивна
  - контролни процеси (Process Control) – конвейер, реално време

Контекстни архитектури (Data Centric)

- характеризират се с централизирано хранилище на данните, които са достъпни за всички компоненти на системата, така че декомпозицията е на модул за управление на достъпа до данните и агенти, които извършват операции върху тях
- интерфейсът между агентите и данните може да е явен – напр. RMI или RPC – или имплицитен – напр. транзактивен
- в чист вид КАрх не предвиждат преки комуникации между информационните агенти – 5.9
- модулът данни изпълнява операции по извличане или регистриране и промяна на записи – по 2 възможни модела:
  - хранилище (repository) – с активни (инициативни) агенти – хранилището е обикн. е организирано като СУБД, CORBA, UDDI или Web-услуги
  - черна дъска – с инициатива на модула данни – агентите са абонати за събития (event listeners), които настъпват при промяна в данните и на които абонатите отговарат реактивно – често при AI-разпределени приложения, охрантелни системи за разпознаване на звук и образ, системи за управление на бизнес ресурси – складове, транспорт

Йерархични архитектури

- декомпозират системата по управление на йерархични модули - т.е. функциите се групират по йерархичен принцип на няколко нива
- координацията обикновено е между модули от различни нива (вертикална свързаност) и се базира на явни (т.е. “заявка-отговор”) обръщения
- ниските нива функционират като услуги към непосредствените по-високи нива; услугите са имплементирани като функции и процедури или пакети от класове
- пълна прозрачност между нивата се постига при запазване на свързващите интерфейси, но имплементацията на услугите може да еволюира
- архитектурен модел на много ОС (Unix, MS .Net) и на протоколните стекове (TCP/IP); разслояване:
  - базови услуги – системните услуги се групират в модули за IO, транзакции, балансирано планиране на процеси, защита на информацията
  - междинен слой – “ядро” – поддържа проблемно-оринетирана логика – бизнес приложения, числова обработка, информационна обработка, като представя интерфейси към колекции от базовите услуги
  - потребителски интерфейсен слой – напр. команден екран, графични контролни прозорци, Shell скрипт интерпретатор

Master/Slaves

- това е вариант на архитектурата с подпрограми, който е специализиран към поддържане на допълнителни нефункционални изисквания – най-вече
  - отказоустойчивост (fault tolerance) и надеждност
  - балансиране за ускорено изпълнение на заявките
- реализира се чрез репликиране на функционалните модули
- задача на M е алтерантивно
  - да оцени адекватността на паралелно обработените резултати от S_n – съществувт протоколи за отказоустойчивост, идентифициращи грешните и верни резултати при ограничен брой на изпълнителните реплики
  - да извърши разпределяне на заявките прилагайки принципите за товарен баланс (л-я 3.)
- блок диаграма и клас диаграма на Master/Slaves архитектура – 6.11

Асинхронни архитектури

- базират се на неявни (implicit) асинхронни обръщения между обслужващите процеси
- асинхронният обмен може да бъде
  - тип Производител /Консуматор (Producer/Consumer) 1:1
    - интерактивен (online) – без буфериране – и двата процеса трябва да са активни, но не блокират изчакващо в точката на обмен
    - отложен (offline) – с опосредяващ обмена процес-буфер на съобщенията; приемащият процес може да не е активен в момента на изпращане на съобщението и обратно
  - тематичен: Издател/Абонати (Publisher/Subscriber) 1:*
    - централизатор Message Topic на всички издадени съобщения и да ги препраща тематично до абонатите – един-към-много обмен

Интерактивни софтуерни архитектури

- поддържат интензивен потребителски интерфейс
- за целта декомпозицията на системата е на 3 функционални модула
  - модул за представяне (изглед) – с потребителски интерфейс – за представяне (в т.ч. графично или мултимедийно) на изходните данни и също намеса на потребителите в обработката (т.е. вход за данни и контрол)
  - модул данни – поддържане на данните с базова функционалност върху тях
  - модул за управление – системни комуникации, управление на процесите, инициализиране и конфигуриране на модули данни, управление на изгледи
- поддържа множество (и то адаптивни) изгледи за даден набор данни
- две категории ИСА: PAC (Presentation-Abstraction-Control) и MVC (Model-View-Controller)
  - аналогията е P-V, A-M и C-C
  - прилагат различно управление:
    - PAC е с йерархично (разслоено) и разпределно управление, при което системата се формира от набор кооперирващи агенти на три нива – базово ниво агенти на общи данни и бизнес логика, ниво на изгледите за локални данни и средно ниво агенти координатори на изгледите; всеки агент интегрира P, A и C компоненти
    - в MVC агентите са равнопоставени

Модели на разпределено обслужване

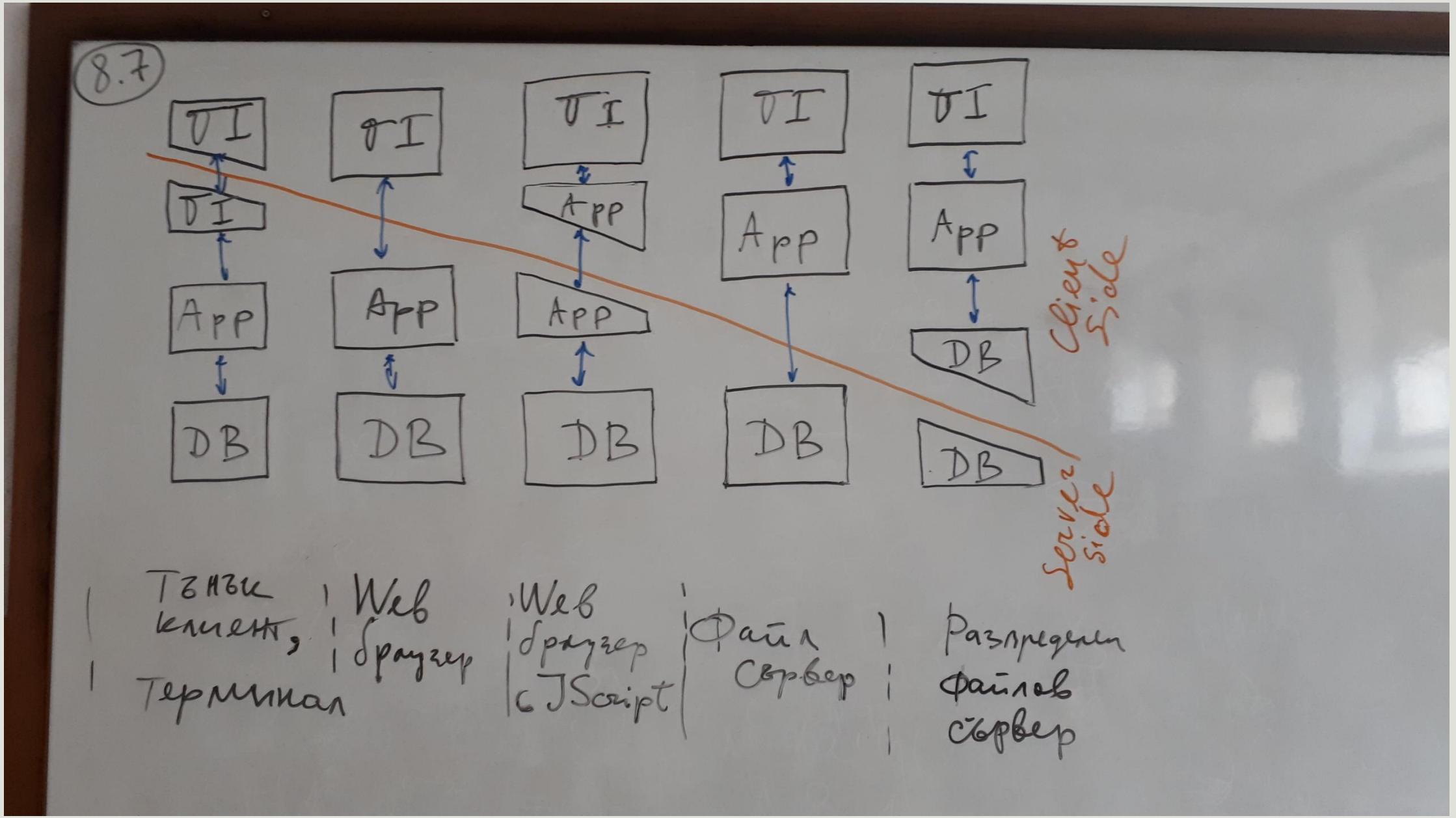
- Сървърни разпределени услуги
  - Клиент-сървер
  - Трислоен модел
  - Брокерен модел
  - Сервизно-базиран модел
  - Компонентно-базиран модел
- Безсървърно обслужване (p2p)

Обхват

- MPMD/MU – за множество потребители (Системи за масово обслужване)
- Характеристики
  - Топология – звезда, дърво/нелинеен конвейер, пълен граф, непълен граф
  - Грануларност
    - декомпозиция на
      - сървера: услуги, компоненти…
      - данните
  - Обмен – синхронен/асинхроннен/срочен (РВ)
  - Управление
    - по време/събитие
    - разпределено /централизирано

Клиент-сървер

- Класическият MPMD-модел на адаптиране на ограничената клиентска инфраструктура за изпълнение на «сложни» и съставни услуги – от тънък до богат клиент, граничещ с безсърверно обслужване (p2p) – активен (инициативен) интерфейсен процес и реактивен процес за приложната логика и данни (фиг. 8.7.)
- Предимства
  - технологично специализиране
  - инфраструктурна гъвкавост (клиентът е с достъп до интерфейси и сензори, сървърът е без такива, но с планирана и проектирана производителност и свързаност)
  - преизползване на сървърните компоненти
  - еволюция на услугите без участието на потребителя
- Недостатъци
  - унификация на клиентската инфраструктура (напр. браузерен достъп)
  - защита на достъпа и данните
  - скалируемост на сървера
  - скъпа поддръжа, профилактика, тестване


8.7
UI
UI
App
DB
UI
App
DB
UI
App
App
DB
UI
App
DB
UI
App
DB
DB
Client side
Server side
Тонкий клиент, Терминал
Web браузер
Web браузер с JScript
Файл сервер
Распределенный файловый сервер

Многослоен сървер

- Декомпозиция на сървера минимум на два слоя (фиг. 8.8.)
  - междинен слой (между клиента – интерфейсния слой и вътрешния слой) за приложната (“business”) логика
  - вътрешен слой – обикновено за достъпа до данни или комуникации
- Предимства на междинния слой
  - технологично специализиране
    - напр. клиентът не генерира SQL-заявки и е напълно прозрачен за сървърната имплементация
  - инфраструкторна гъвкавост
    - напр. изтегляне и независимост на трафика от [ненадеждната] клиентска инфраструктура
  - преизползване на сървърните компоненти
  - еволюция на услугите без [сложно] да се еволюират данните – напр.:
    - многонишкова междинна услуга за ускорение или скалируемост
    - рекурсивен сървер с нова нишка за всяка нова заявка и/ли потребител като Apache –
    - итеративен сървер с изпълнение на конкурентните заявки една по една от опашка като Xampp – практичен при малки и балансирани натоварвания
- Недостатъци
  - унификация на клиентската инфраструктура (напр. браузерен достъп)
  - защита на достъпа и данните
  - скалируемост на сървера
  - поддръжка, профилактика, тестване – в още по-голяма степен

Брокер на услуги

- Брокерът е инстанция на междинния слой, която поддържа достъпът до и обработката на множество услуги или алтернатвни услуги или външни услуги (фиг. 8.9.) чрез:
  - à la директория на услугите
  - контекстно (т.е. приложно) класиране на услугите – функционално, а и нефункционално
  - динамично адресиране (прозрачно, по параметър – look-up) освен статичното при [3t]C-S (по адрес или име, но които са с уникалност на идентификатор)
- Функции
  - регистриране на услугите и актуализация на регистъра
    - обикновено с публикуване на заредим от клиента интерфейс на услугите (напр. бисквити и проксита)
  - достъп на клиентските процеси
  - класиране и препращане на клиентските заявки към една или повече услуги
  - връщане на генериран резултат или обработка на изключението
- Обменът се базира на клиентско и брокерно прокси – допълнителен протокол (т.е. резидентен процес), обикновено изпълняващ функциите на [де-]сериализиране ([un-]marshalling) и други представителни преобразовния

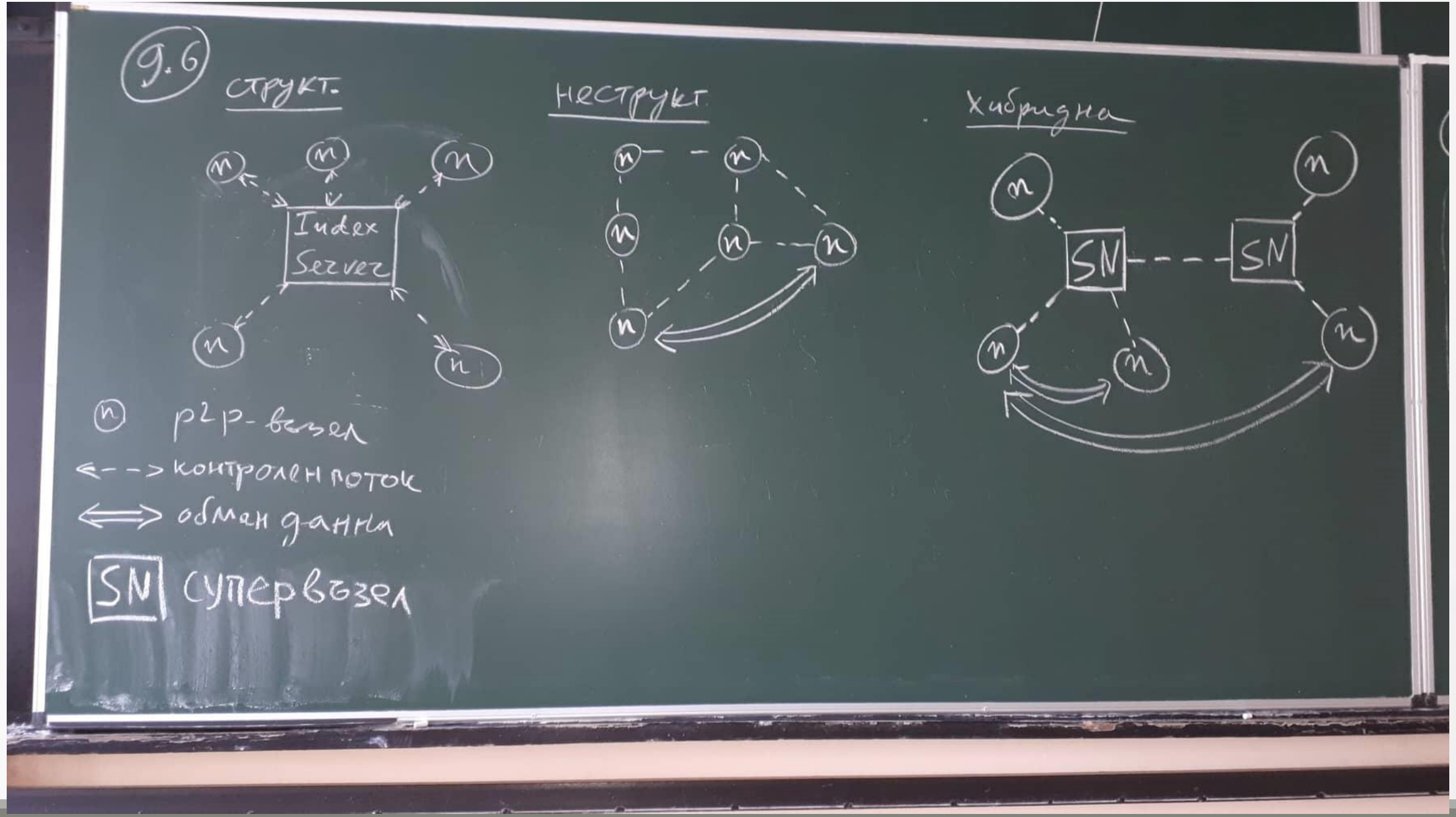
Безсърверно масово обслужване ≈ p2p

- Особености
  - разгръща се като наслоена мрежа (overlay) върху съществуващата клиентска част от IT-инфраструктурата и поддържа нейерархичен модел на обмен върху йерархичния IP; без участие и използване на трафик в опорните мрежи (backbone) и техните възли (приложни и комуникационни сървери)
  - децентрализирана топология – равнопоставени процеси с универсална специализация – едновременно клиентски функции и частични сървърни функции (напр. [малка] част от данните за единичен пряк или групов и дори множествен достъп; маршрутизиране на съобщения между външни процеси и т.н.)
  - самоорганизация – без единен процес на управление, маршрутизация, регистрация на ресурси и данни; с динамични и вариращи параметри на ресурсите, на адресите на обекти; възможно с частична сървърна поддръжка за наблюдение и управление
  - инцидентна (ad hoc) и динамична свързаност и наличност на възлите и процесите в тях, но с изисквания за скалируемост
  - скалируемост (разширимост) – присъща поради отсъствието на централизиран топологичен елемент като сървера (тясно място)
  - отказоустойчивост (fault tolerance) – присъща поради множеството обслужващи ресурси, но с необходимост от репликиране на критичните данни и услуги
  - анонимност на споделянето – възлите като ресурс и свързаните процеси в тях се използват за транспорт и други системни функции (например репликиране на критични данни – „кеширане“) без конкретно оторизиране от администратора на възела или на процеса или на данните, но със свободно доброволно участие на потребителите
  - специална юридическа регулация

Типове



- Наслоени мрежи за БсМО:
  - неструктурирани – произволен граф на свързаност и производителност на комуникационните канали, спонтанно добавяне и отпадане на възли (типично за нововъзникнали БсМО)
  - структурирани – проектирани (а не спонтанни) характеристики:
    - топология – типично кръг, хорда (с „пръсти“ за вс. възел), дърво, верига;
    - комуникационна производителност;
    - ограничения в броя възли и тяхната дистанция (по време)
  - хибридни и йерархични структурирани – неструктурирана супермрежа от супервъзли, участващи в по 1+ структурирана наслоена мрежа
- Разпределени хеш-таблици (DHT) – за безсърверни данни


9.6
структ.
Index Server
неструкт.
Хибридна
SN
SN
p2p-възел
контролен поток
обмен данни
SN супервъзел

Синхронизация с времеви марки (timestamps)

- (Lamport, 1978): синхронизиращи съобщения между възлите с времеви марки на локалните логически времена
- ако получаващия процес има по-малка стойност на локалното логическо време от марката на изпретеното съобщение, той коригира своя логически часовник (само в положителна посока!) към стойност (марка+1) – фиг. 7.23.
- изискване: няма две събития с еднакво C – ако синхронизиращия процес изпраща/приема едновременно две съобщения с времеви марки, тои ги дистанцира логически на 1 такт
- допълнително прецизиране на логическото време (за уникални марки) се постига като към целочислената марка се добави процесния идентификатор (или негова производна) като дробна част

Протокол за тотално подреждане

- прилага логическа синхронизация с времеви марки за еднакво подреждане на събитията (получаване на съобщения) при групово предаване (multicasting) – напр. при коригиране на записите в репликирана база данни
- при групово предаване на съобщения с времеви марки изпращащия процес като член на групата получава своите съобщения и то в реда на изпращането им и без загуби
- всеки приемащ процес записва получените съобщения в локален буфер по реда на времевите марки и потвърждава приемането до процесите в групата; потвържденията също се маркират (дистанцирано от съотв. съобщение)
- същевремнно се прилага и алгоритъма на Лампорт за положителни корекции на локалното логическо време
- всички съобщения – вкл. потвържденията! – са групови (независимо дали са предназаначени за всички процеси в групата)
- локалните буфери са опашки (FCFS) от които съобщенията се предават към съответните локални приложения, като се изтриват от буфера (както и техните потвърждения)
- резултат: всички локални буфери са с еднакво подреждане на съобщенията и потока от съобщения към всяко локално приложение е идентичен (N.B.: еднаквото подреждане обаче не гарантира запазване на реда на възникване на събитията в реално астрономическо време → алгоритъма на Лампорт е приложим за събития, между които няма причинно-следствена връзка – causality)

Протокол за съхранено подреждане

- позволява тотално подреждане на събития при запазване на реда им в реално време – напр. при публикуване на дискусионни и новинарски бюлетини, където е важна не само идентична подредба, но и запазване на причинно-следствената връзка – т.е. съхранено подреждане (causally ordering)
- прилага векторна маркировка (vector timestamp):
- всеки процес P_i поддържа свой вектор от броячи V_i, чиито елементи отразяват броя събития, настъпили в процесите с съответен индекс – $V_i[j]$ = брой настъпили събития в P_j; $V_i[i]$ = брой събития в P_i
- за целта когато P_i изпраща съобщението m, към него добавя (т.нар. piggybacking) и текущата стойност на своята вектор V_i като векторна марка vt
- по този начин получаващият съобщението m процес P_j е информиран за броя събития, възникнали във всички процеси преди P_i да изпрати m – т.е. общия брой събития, от които изпращането на m може (потенциално) да е следствие
- при получаването на m P_j прави корекциите $V_j[k] = \max\{V_j[k], vt[k]\}$ и $V_j[i]$++, при което P_j вече разполага с броя събития-съобщения, които предхождат (евентуално като причина) m (и съответно – ако има такива – може да ги изчака)

Вложени транзакции

- вложени (nested) транзакции – представляват йерархичен дървовиден набор от субтранзакции, първата от които инициира няколко от следващото ниво и т.н. – в съответствие с логическото и каузално (причинно-следствено) разделение на цялата “супертранзакция”; всяка от субтранзакциите е логически независима от изпълнението на останалите (примера с последователните полети – фиг. 7.26.)
- целта е да се постигне ускорено изпълнение при паралелно изпълнение от няколко сървера, но може да се ползват и за съхраняваняване на междинни резултати
- наборът субтранзакции се счита за изпълнен, само ако главната субтранзакция е изпълнена, а ако не е – заличават се и резултатите на успешно изпълнените дъщерни субтранзакции (което може да породи проблем особено при изпълнение в РС)
- изпълнението на вложените транзакции е рекурсивно: когато главната субтранзакция е изпълнена, за изпълнени се считат и другите завършили субтранзакции по йерархията; резултатите от неизпълнените субтранзакции се заличават

Разпределени транзакции

- при тях декомпозицията на супертранзакцията в субтранзакции не следва логическото разделение, а се определя от структурата на разпределения контекст – напр. разпределна база данни, върху всеки от дяловете на която оперира отделна субтранзакция
- пример: междубанков трансфер със субтранзакции върху различни бази данни – фиг. 7.27.
- контраст с блоковите транзакции: блокова е напр. транзакция за начисляване на лихва по сметка (в една база данни)